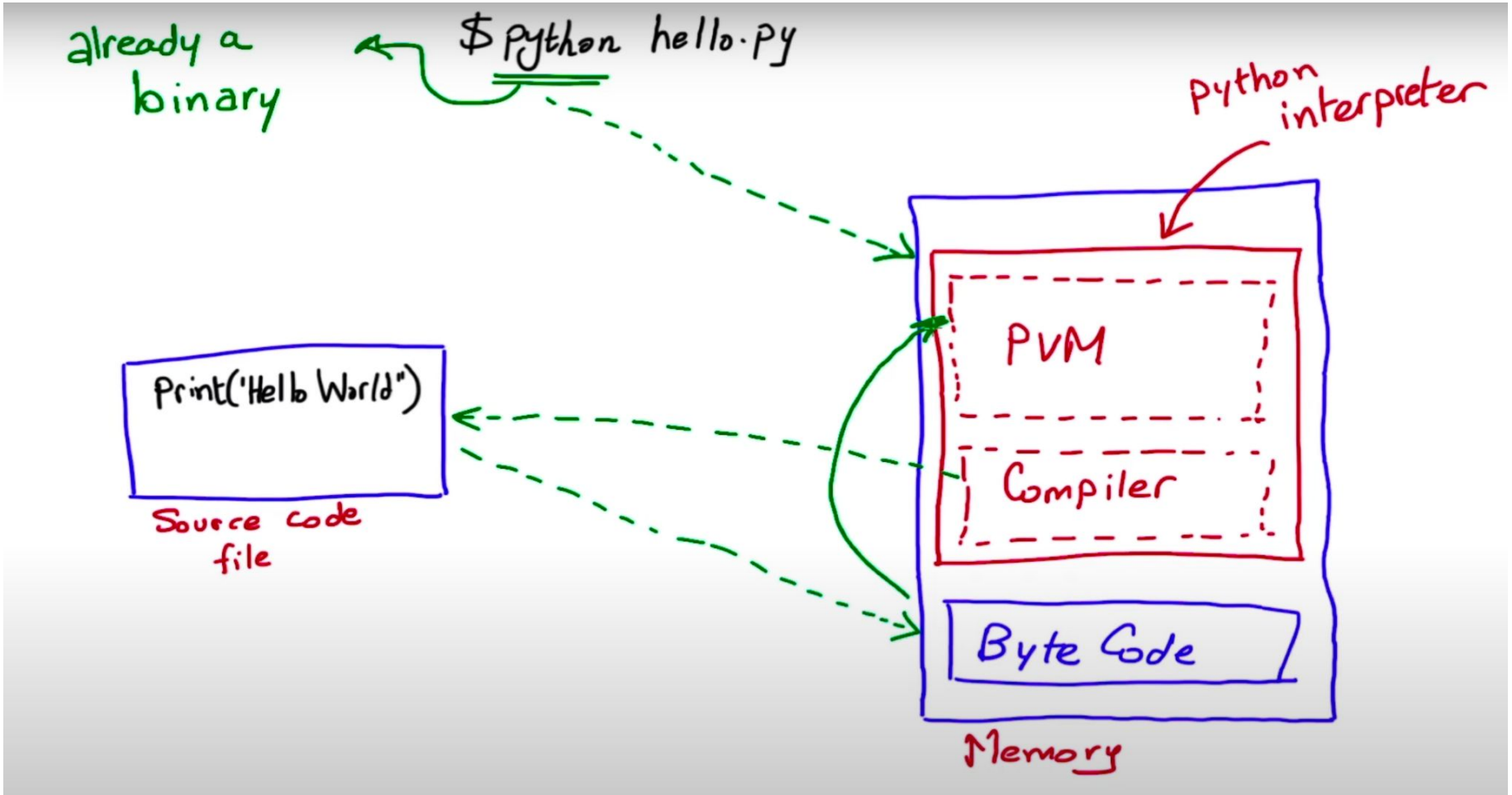
MDL - Tutorial

Machine Learning with Python - scikit learn

Python - DYNAMIC INTERPRETER



PYTHON - NUMPY

- Python Library,adding support for large,multidimensional array and matrices(and other mathematical functions).

- NumPy arrays are much faster and have strict requirements on the homogeneity of the objects.

- Computations on arrays can be written very efficiently in a low-level language such as C(and a large part of Numpy is actually written in C). Knowing the address of the memory block and the data type, it is just simple arithmetic to access an item for example. There would be significant overhead to do that in Python with a list.

- Spatial locality in memory access patterns results in performance gains notably due to the CPU cache.

PREPARING YOUR DATA

- In scikit-learn :

-> X is a (n,p) numpy array n input observations in dimension p.

-> y is a (n,p_out) numpy array expected outputs.

- Pre-Processing :

-> Use Transformers (next slide)

TRANSFORMERS

When building machine learning pipelines in `scikit-learn`, we often need to perform data transformations.

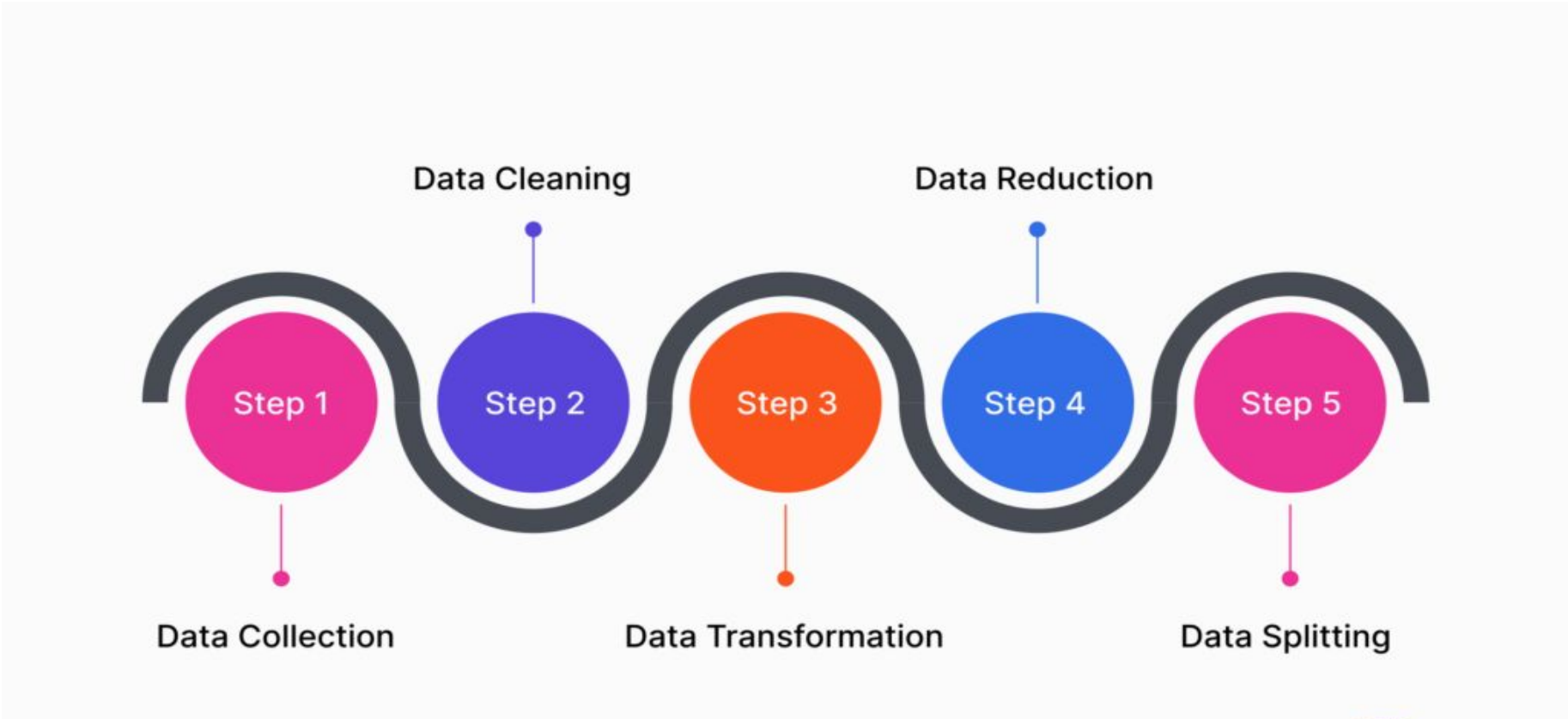
Transformers help in:

- Feature scaling (e.g., Standardization, Normalization)
- Feature encoding (e.g., One-hot encoding, Label encoding)
- Dimensionality reduction (e.g., PCA)
- Feature engineering (e.g., Discretization, Binning)

Example

```
>>> from sklearn.preprocessing import StandardScaler
>>> data = [[0, 0], [0, 0], [1, 1], [1, 1]]
>>> scaler = StandardScaler()
>>> scaler.fit(data)
>>> print(scaler.mean_)
[0.5 0.5]
>>> print(scaler.transform(data))
[[-1. -1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1.  1.]]
>>> print(scaler.transform([[2, 2]]))
[[3. 3.]]
```

Developing your ML model - STEPS



SCIKIT LEARN - FORMAT OF THE DATA

```python
# Define X as a (n, p) NumPy array (5 samples, 3 features)
X = np.array([
    [25, 50000, 1],
    [32, 60000, 0],
    [40, 70000, 1],
    [29, 55000, 0],
    [35, 65000, 1]
])

# Define y as a (n, p_out) NumPy array (5 samples, 1 output)
y = np.array([
    [0],
    [1],
    [0],
    [1],
    [1]
])

model = LinearRegression()
model.fit(X, y)

X_test = np.array([
    [25, 50000, 1],
    [32, 60000, 0],
    [40, 70000, 1],
    [29, 55000, 0],
    [35, 65000, 1]
])

y_pred = model.predict(X_test)

print("Predicted y:\n", y_pred)
```

- Data is typically formatted as a NumPy array where each row represents a data sample and each column represents a feature, with the target values (labels) stored separately as a separate array.
- Example for splitting dataset into train and test :

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

CONSTRAINTS ON FEATURES

Features can be :

1. Real values
2. Integer values to represent categorical data

If you have strings in your data, you first have to convert them to integers (Use Transformers).

Example : One Hot Encoding

```python
# Sample categorical data (3 samples, 1 categorical feature)
X = np.array([
    ["Red"],
    ["Blue"],
    ["Green"]
])

# Create OneHotEncoder
encoder = OneHotEncoder(sparse_output=False)  # Set sparse_output=False to get a dense NumPy array

# Fit and transform the data
X_encoded = encoder.fit_transform(X)

print("Original X:\n", X)
print("\nOne-Hot Encoded X:\n", X_encoded)
print("\nFeature Categories:", encoder.categories_)
```

EMS OUTPUT COMMENTS PORTS

TERMINAL

```
(3.8.12) Mohammads-MacBook-Air:Coding zaid$ python3 sample.py
Original X:
 [['Red']
 ['Blue']
 ['Green']]

One-Hot Encoded X:
 [[0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
```

California HOUSING DATASET

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
14196	3.2596	33.0	5.017657	1.006421	2300.0	3.691814	32.71	-117.03
8267	3.8125	49.0	4.473545	1.041005	1314.0	1.738095	33.77	-118.16
17445	4.1563	4.0	5.645833	0.985119	915.0	2.723214	34.66	-120.48
14265	1.9425	36.0	4.002817	1.033803	1418.0	3.994366	32.69	-117.11
2271	3.5542	43.0	6.268421	1.134211	874.0	2.300000	36.78	-119.80
17848	6.6227	20.0	6.282147	1.008739	2695.0	3.364544	37.42	-121.86
6252	2.5192	28.0	4.345361	1.074742	1355.0	3.492268	34.04	-117.97
9389	7.9892	37.0	6.052758	0.954436	999.0	2.395683	37.91	-122.53
6113	1.5000	5.0	3.620579	1.016077	819.0	2.633441	34.13	-117.90
6061	6.4266	5.0	6.730284	1.030609	9427.0	3.394671	34.02	-117.79

1.03
3.821
1.726
0.934
0.965
2.648
1.573
5.00001
1.398
3.156

The target variable is the median house value for California districts, expressed in hundreds of thousands of dollars($100,000)

sample.py

```python
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.metrics import mean_squared_error, r2_score

housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = housing.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

selector = SelectKBest(f_regression, k=5)
X_train_selected = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

model = LinearRegression()
model.fit(X_train_selected, y_train)

y_pred = model.predict(X_test_selected)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)

selected_features = X.columns[selector.get_support()]
print("Selected Features:", selected_features)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Mohammads-MacBook-Air:Coding zaid$ python3 sample.py
Mean Squared Error: 0.6382565441555914
R-squared: 0.5129333248216976
Selected Features: Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Latitude'], dtype='object')
```

Can we do better?

THANK YOU!